

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-23T11:49:16.3401736Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260923-114916369.prof
Profil mémoire	memory-20260923-114916369.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	360.40 ms	360395676	3870845	24

Détail hyperfine

Métrique	Valeur
Moyenne	36.9659 s
Écart-type	n/a
Médiane	36.9659 s
Temps utilisateur	35.6191 s
Temps système	1.5375 s
Min	36.9659 s
Max	36.9659 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	36.43s
Mémoire allouée (pprof)	388.06MB
Mémoire par opération	3870845 B/op

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **36.43s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
----------	------	--------	--------	----------

jeu-de-la-vie/game.(*Board).shouldInfect	25.08s	68.84%	35.84s	98.38%
jeu-de-la-vie/game.(*Board).coordinates (inlin	5.80s	15.92%	5.80s	15.92%
jeu-de-la-vie/game.squaredDistance (inline)	4.96s	13.62%	4.96s	13.62%
jeu-de-la-vie/game.(*Board).Step	0.12s	0.33%	36.09s	99.07%
jeu-de-la-vie/game.BenchmarkTick	0	0.00%	36.09s	99.07%
testing.(*B).launch	0	0.00%	36.09s	99.07%
testing.(*B).runN	0	0.00%	36.09s	99.07%

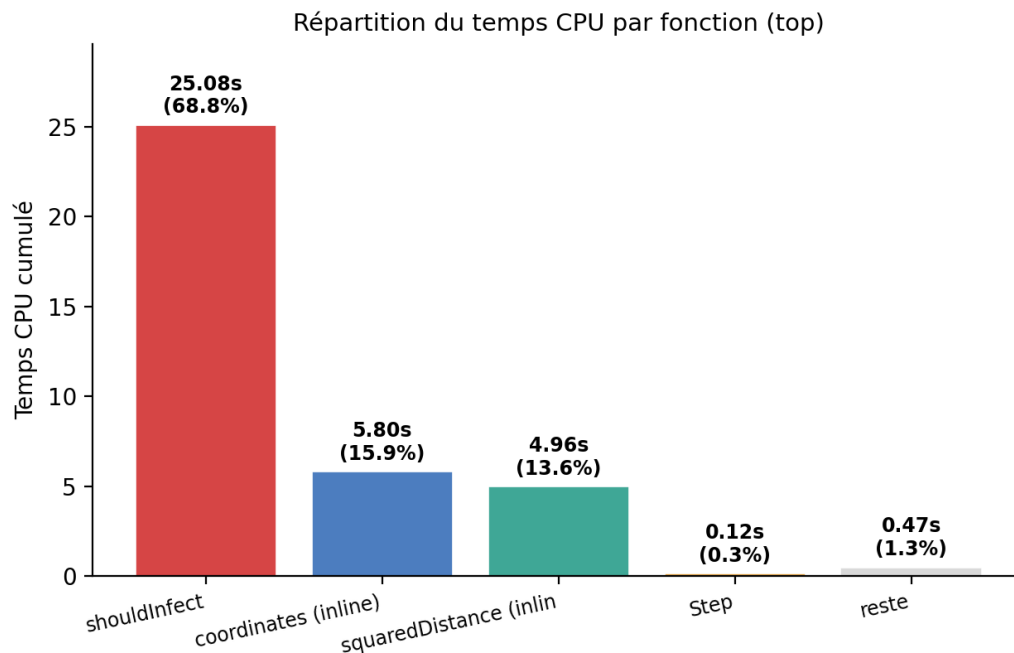


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

(Graphe d'appel visuel non généré — Graphviz/'dot' introuvable dans le PATH. Installez Graphviz pour l'obtenir automatiquement au prochain run.)

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).shouldInfect**, responsable de **68.8%** du temps CPU total (flat), pour **98.4%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

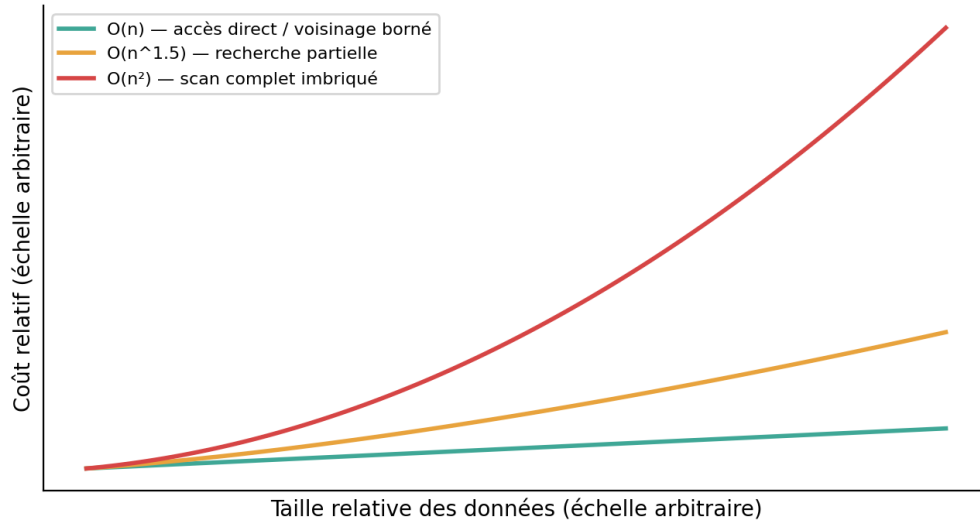


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **388.06MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).Step	374.24MB	96.44%	374.24MB	96.44%
jeu-de-la-vie/game.RandomBoard	5.52MB	1.42%	6.21MB	1.60%
jeu-de-la-vie/game.BenchmarkTick	0	0.00%	380.46MB	98.04%
main.main	0	0.00%	2.22MB	0.57%
runtime.main	0	0.00%	2.72MB	0.70%
runtime/pprof.(*profileBuilder).readMapping	0	0.00%	2MB	0.52%
runtime/pprof.newProfileBuilder	0	0.00%	2MB	0.52%
runtime/pprof.peBuildID	0	0.00%	2MB	0.52%

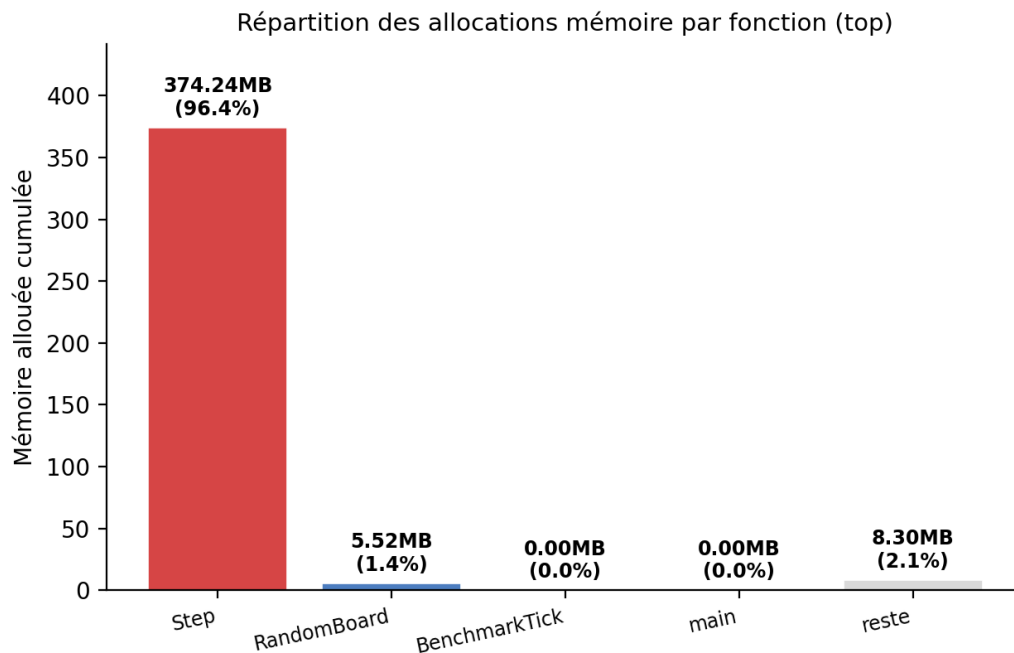


Figure — Répartition des allocations mémoire par fonction.

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé : ≥ 10 runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.
- **Pas de trace GC détaillée** (GODEBUG=gctrace=1) — utile si les allocations mémoire (section 3) sont significatives.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- Les métriques CPU bas niveau, GC détaillé, I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-23 14:01:02 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.